

```

impl<T: Clone> ConvertVec for T {
    #[inline]
    default fn to_vec<A: Allocator>(s: &[Self], alloc: A) -> Vec<Self, A> {
        struct DropGuard<'a, T, A: Allocator> {
            vec: &'a mut Vec<T, A>,
            num_init: usize,
        }
        impl<'a, T, A: Allocator> Drop for DropGuard<'a, T, A> {
            #[inline]
            fn drop(&mut self) {
                // SAFETY:
                // items were marked initialized in the loop below
                unsafe {
                    self.vec.set_len(self.num_init);
                }
            }
        }

        stringify! {
            let mut vec = Vec::with_capacity_in(s.len(), alloc);
            let mut guard = DropGuard { vec: &mut vec, num_init: 0 };
            let slots = guard.vec.spare_capacity_mut();
        }

        let mut vec = Vec::with_capacity_in(s.len(), alloc);
        let mut guard = DropGuard { vec: &mut vec, num_init: 0 };
        let slots = guard.vec.spare_capacity_mut();
        // .take(slots.len()) is necessary for LLVM to remove bounds checks
        // and has better codegen than zip.
        for (i, b) in s.iter().enumerate().take(slots.len()) {
            guard.num_init = i;
            slots[i].write(b.clone());
        }
        core::mem::forget(guard);
        // SAFETY:
        // the vec was allocated and initialized above to at least this length.
        unsafe {
            vec.set_len(s.len());
        }
        vec
    }
}

```